Step-by-Step: Own DevOps Project — Real-Time Style

1️⃣ Simple Application ante enti?

Answer:

DevOps project start cheyyadaniki first manaki oka **application** kavali.

Application ante user use chese software.

Examples:

To-Do Application

Static Website

Node.js App

Python Flask App

Java Spring Boot App

Simple ga:

"DevOps practice cheyyadaniki first deploy cheyyadaniki oka application kavali."

Enduku use cheyali?

DevOps lo manam application ni:

Code

↓

Build

↓

Test

↓

Docker

↓

CI/CD

↓

Cloud

↓

Deploy

↓

Monitor

chesi practice chestham.

Ela use cheyali?

Example ga simple Node.js application teesukundam:

```
myapp/
│
├── app.js
├── package.json
└── README.md
```

Application local ga run cheyyali:

```
npm install
npm start
```

Application running:

User

↓

Node.js Application

↓

Response

Issue vaste ela fix cheyali?

Application run avvakapothe:

```
npm install
npm start
```

logs check cheyyali.

Error

↓

Read error message

↓

Check dependencies

↓

Check configuration

↓

Fix code

↓

Run again

Real-time example:

Company ki **online shopping application** undi.

First:

Shopping App

↓

Developer code

↓

Local testing
Tarvata ee application ni DevOps pipeline lo deploy chestham.

2️⃣ Git & GitHub ante enti?
Answer:
Git ante code changes ni track cheyyadaniki use chese **version control system**.
GitHub ante Git repositories ni online lo store/manage cheyyadaniki use chese platform.
Simple ga:
"Developer code changes ni safely track chesi, team tho collaborate cheyyadaniki Git and GitHub use chestham."
Enduku use cheyali?
Without Git:
app-final
app-final-new
app-final-new2
app-final-working
ila files manage cheyyadam kastam.
Git tho:
Version 1
↓
Version 2
↓
Version 3
Every change track cheyyachu.
Ela use cheyali?
First repository create cheyyali.
git init
Files check:
git status
Files add:
git add .
Commit:
git commit -m "Initial commit"
GitHub repository ki connect chesi:
git remote add origin <repo-url>
git push -u origin main
Flow:
Developer
↓
Code change
↓
git add
↓
git commit
↓
git push
↓
GitHub
Issue vaste ela fix cheyali?
GitHub ki push avvakapothe:
git status
git remote -v
git branch
check cheyyali.
Wrong branch ayithe:
git branch
git checkout main
Tarvata push cheyyali.
Real-time example:
Developer login feature add chesthadu:
Developer
↓
Code change
↓
git add .
↓
git commit
↓
git push
↓
GitHub
Ippudu CI/CD pipeline automatically trigger avvachu.

3️⃣ **Docker ante enti?**
Answer:
Docker ante application ni **container image** ga package chesi, same environment lo run cheyyadaniki use chese containerization platform.
Simple ga:
"Application + dependencies ni oka container image lo package chesi ekkadaina consistent ga run cheyyadaniki Docker use chestham."
Enduku use cheyali?
Developer machine lo:
"It works on my machine"
Production lo:
❌ Application not working
Environment differences valla problems vastayi.
Docker tho:
Application
+
Dependencies
+
Runtime
↓
Docker Image
↓
Container
Ela use cheyali?
First Dockerfile create chestham.
Example:

```
FROM nginx

COPY . /usr/share/nginx/html

EXPOSE 80
```

Image build:

```
docker build -t my-app .
```

Container run:

```
docker run -p 8080:80 my-app
```

Flow:
Application
↓
Dockerfile
↓
docker build
↓
Docker Image
↓
docker run
↓
Container
↓
Application Running
Issue vaste ela fix cheyali?
Container run avvakapothe:

```
docker ps
docker ps -a
docker logs <container-id>
```

Image check:

```
docker images
```

Port problem check cheyyali:
Host Port → Container Port
Real-time example:
Developer Node.js app create chesadu.
Node.js App
↓
Dockerfile
↓
docker build
↓
my-app:v1
↓
docker run
↓

Container
↓
Application
Ippudu same image testing and production environments lo use cheyyachu.

CI/CD Pipeline ante enti?
Answer:
CI/CD ante application code changes ni **automatically build, test, package, and deploy cheyyadaniki use chese automated pipeline**.
Simple ga:
"Developer code push chesina tarvata build, test, Docker image creation and deployment automatic ga jaragadaniki CI/CD use chestham."
Enduku use cheyali?
Without CI/CD:
Developer
↓
Manual build
↓
Manual test
↓
Manual Docker build
↓
Manual deployment
Mistakes ekkuva.
CI/CD tho:
Code Push
↓
Build
↓
Test
↓
Docker Image
↓
Push Registry
↓
Deploy
Ela use cheyali?
GitHub Actions example:

```
name: CI Pipeline

on: [push]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v2

    - name: Build Docker image
      run: docker build -t my-app .
```

Flow:
Developer
↓
git push
↓
GitHub
↓
GitHub Actions
↓
Build
↓
Test
↓
Docker Image
Issue vaste ela fix cheyali?
Pipeline fail ayithe:
GitHub
↓
Actions
↓

Failed Job
↓
Check logs
↓
Identify failed step
↓
Fix code/YAML
↓
Push again

Real-time example:

Developer bug fix chesthadu:

```
git add .
git commit -m "Fix login bug"
git push
```

Immediately:

GitHub
↓
CI/CD Trigger
↓
Build
↓
Test
↓
Docker Build
↓
Deploy

5️⃣ Terraform ante enti?

Answer:

Terraform ante **Infrastructure as Code (IaC)** tool.

Cloud infrastructure ni manually create cheyyakunda **code/configuration files dwara create and manage cheyyadaniki Terraform use chestham.**

Simple ga:

"AWS lo servers, networks, storage lanti infrastructure ni manual ga create cheyyakunda code dwara create cheyyadaniki Terraform use chestham."

Enduku use cheyali?

Manual ga:

AWS Console
↓
EC2 create
↓
VPC create
↓
Security Group
↓
S3

Time-consuming.

Terraform tho:

Terraform Code
↓
terraform plan
↓
terraform apply
↓
AWS Infrastructure

Ela use cheyali?

Example:

```
resource "aws_instance" "web" {
  ami           = "ami-0abcdef1234567890"
  instance_type = "t2.micro"
}
```

Commands:

```
terraform init
```

Then:

```
terraform plan
```

Then:

```
terraform apply
```

Flow:

Terraform Code
↓
terraform init

↓
terraform plan
↓
terraform apply
↓
AWS
↓
EC2 Created
Issue vaste ela fix cheyali?
First:
terraform plan
error check cheyyali.
State/configuration issues kosam:
terraform validate
terraform plan
check cheyyali.
Real-time example:
Company ki new application kosam server kavali.
Developer manually EC2 create cheyyakunda:
Terraform
↓
EC2
↓
VPC
↓
Security Group
↓
Infrastructure Ready

💧 **Ansible ante enti?**
Answer:
Ansible ante servers ni **configure and manage cheyyadaniki use chese configuration management/automation tool**.
Simple ga:
"Multiple servers lo same software and configuration ni automatic ga setup cheyyadaniki Ansible use chestham."
Enduku use cheyali?
Suppose 50 servers unnayi.
Manual ga:
Server-1 → Nginx install
Server-2 → Nginx install
Server-3 → Nginx install
...
Server-50 → Nginx install
Chaala time.
Ansible tho:
Ansible Playbook
↓
50 Servers
↓
Nginx installed
Ela use cheyali?
Example:

```
- name: Setup Web Server
  hosts: web
  become: yes

  tasks:
    - name: Install Nginx
      apt:
        name: nginx
        state: present
```

Run:
ansible-playbook setup.yml
Flow:
Ansible Playbook
↓
Ansible
↓
Server-1
Server-2
Server-3
↓

Nginx Installed
Issue vaste ela fix cheyali?
Check connectivity:
ansible all -m ping
Playbook error vaste output/log chusi:
Connection
↓
Permissions
↓
Package name
↓
Configuration
check cheyyali.
Real-time example:
Company ki 20 web servers unnayi.
Ansible
↓
20 Servers
↓
Install Nginx
↓
Copy configuration
↓
Start Nginx
Oka command tho complete setup.

7️⃣ Cloud Deployment ante enti?
Answer:
Cloud deployment ante application ni **AWS/Azure/GCP infrastructure lo deploy chesi users ki available cheyyadam**.
Simple ga:
"Local machine lo unna application ni cloud servers lo deploy chesi internet users ki available cheyyadam."
Enduku use cheyali?
Local machine:
Developer Laptop
Only developer access cheyyagaladu.
Cloud lo:
Internet
↓
Cloud
↓
Application
Users anywhere nundi access cheyyachu.
Ela use cheyali?
Example AWS:
Application
↓
Docker Image
↓
AWS EC2 / EKS
↓
Load Balancer
↓
Users
Issue vaste ela fix cheyali?
Application access avvakapothe:
Application status
↓
Server status
↓
Security Group
↓
Port
↓
Load Balancer
↓
DNS
check cheyyali.
Real-time example:
User
↓
DNS

↓
AWS Load Balancer
↓
EC2 / EKS
↓
Docker Container
↓
Application

8️⃣ Monitoring & Alerting ante enti?
Answer:
Monitoring ante application and infrastructure **healthy ga unnaya, resources ela use avuthunnayi ani continuously observe cheyyadam**.
Alerting ante problem vachinappudu team ki notification pampinchadam.
Simple ga:
"Production application health and performance ni continuously monitor chesi, problem vaste alert ivvadaniki monitoring use chestham."
Enduku use cheyali?
Production lo issue vachinappudu user complaint kosam wait cheyyakudadhu.
Example:
CPU = 95%
↓
Monitoring detects
↓
Alert
↓
DevOps team
↓
Issue investigate
Ela use cheyali?
Tools:
AWS CloudWatch
Prometheus
Grafana
Alertmanager
Example Kubernetes:
kubectl top pods
kubectl top nodes
Issue vaste ela fix cheyali?
Example:
CPU High
↓
Check Pod
↓
kubectl top pods
↓
kubectl describe pod
↓
kubectl logs
↓
Find issue
↓
Fix
Real-time example:
Production application lo traffic sudden ga increase ayyindi:
Traffic Increase
↓
CPU 90%
↓
Prometheus detects
↓
Grafana shows metric
↓
Alertmanager sends alert
↓
DevOps investigates
↓
Scale application

9️⃣ Complete Real-Time DevOps Project Flow
Answer:

Suppose mana company ki **Node.js online shopping application** undi.
Developer first application code create chesthadu:
Node.js Application
Then GitHub lo push chesthadu:
Code
↓
Git
↓
GitHub
Docker tho application containerize chestham:
GitHub
↓
Dockerfile
↓
Docker Image
↓
Container
CI/CD pipeline trigger avuthadi:
Git Push
↓
GitHub Actions / Jenkins
↓
Build
↓
Test
↓
Docker Image
↓
Docker Registry
Terraform infrastructure create chesthadi:
Terraform
↓
AWS
↓
VPC
↓
EC2 / EKS
↓
Networking
Ansible server configuration chesthadi where applicable:
Ansible
↓
Server
↓
Install packages
↓
Configure application
Application cloud lo deploy avuthadi:
Docker Image
↓
AWS / Kubernetes
↓
Application Running
Monitoring start avuthadi:
Application
↓
Prometheus / CloudWatch
↓
Grafana
↓
Alerts
🔥 **Complete flow:**
Developer
↓
Code
↓
Git
↓
GitHub
↓
CI/CD

↓
Build + Test
↓
Docker Image
↓
Docker Registry
↓
Terraform
↓
Cloud Infrastructure
↓
Ansible / Kubernetes
↓
Application Deployment
↓
Load Balancer
↓
Users
↓
Monitoring
↓
Alerts
↓
Troubleshooting

🔥 **Real-time problem example**

Suppose production lo application down ayindi.

DevOps engineer flow:

Monitoring Alert
↓
Check application
↓
kubectl / cloud commands
↓
Check logs
↓
Check container
↓
Check server
↓
Check networking
↓
Find root cause
↓
Fix
↓
Deploy corrected version
↓
Monitor again

👉 **Idhe real-world DevOps working cycle:**

Code → Git → CI/CD → Docker → Infrastructure → Deployment → Monitoring → Troubleshooting → Fix → Redeploy.